

Propuesta de Proyecto: BiblioFlow - Sistema de Gestión Bibliotecaria Integral

1. Visión General del Proyecto

El presente documento formaliza la asignación y define el alcance del proyecto **BiblioFlow**, una plataforma integral diseñada para modernizar la gestión de bibliotecas, centros de documentación y archivos. BiblioFlow nace de la necesidad de transformar la operativa tradicional, basada en sistemas monolíticos obsoletos, catálogos de fichas o software desconectado, hacia un ecosistema digital centralizado, accesible y eficiente. El objetivo es ofrecer una solución completa que abarque desde la catalogación de fondos hasta la experiencia del usuario final, facilitando el préstamo, la consulta y la gestión de colecciones.

La plataforma BiblioFlow no es solo un sistema de préstamos, sino un **ecosistema integral de gestión bibliotecaria** que cubre el ciclo de vida completo de los materiales: desde su adquisición y catalogación, pasando por su disponibilidad para préstamo o consulta en sala, hasta su mantenimiento y eventual baja. Dada la naturaleza variada de los fondos (libros, revistas, material audiovisual, mapas, fondos de archivo) y la necesidad de ofrecer servicios tanto al público (socios) como al personal bibliotecario, el sistema se construirá bajo una **arquitectura de microservicios flexible y escalable**. Cada dominio de negocio (catálogo, circulación, usuarios, adquisiciones) operará como un servicio independiente pero perfectamente integrado a través de un API Gateway. El resultado será una herramienta que no solo automatice las tareas diarias de la biblioteca, sino que mejore la experiencia de los socios mediante un portal de consulta y autogestión, y proporcione a los bibliotecarios la visibilidad necesaria para optimizar la colección y los servicios.

2. Funcionalidades Clave del Sistema

- **Microservicio de Gestión de Socios y Usuarios (Patron Service):** Gestiona los perfiles de los socios de la biblioteca, así como del personal bibliotecario. Almacena datos personales, historial de préstamos, multas o sanciones pendientes, y preferencias de comunicación. Implementa un sistema de roles: **administrador** (gestión global), **bibliotecario** (préstamos, devoluciones), **socio** (consulta y autogestión). Soporta autenticación mediante credenciales y posible integración con sistemas de identidad externos (carné universitario, DNIe).
- **Microservicio de Catálogo Bibliográfico (Catalog Service):** El corazón de la plataforma. Gestiona el registro bibliográfico de todos los materiales de la biblioteca siguiendo estándares internacionales como **MARC21** o **Dublin**

Core. Almacena metadatos detallados: título, autor, editorial, ISBN/ISSN, año, edición, materia, resumen, e imágenes de portada. Soporta la creación de **autoridades** (autores, materias normalizadas) para garantizar la consistencia del catálogo. Permite búsquedas avanzadas y facetadas.

- **Microservicio de Gestión de Ejemplares (Holdings Service):** Gestiona los ejemplares físicos o digitales asociados a cada registro bibliográfico. Un libro (registro) puede tener múltiples ejemplares (copias) en diferentes ubicaciones (salas, depósitos) y con diferentes estados (disponible, prestado, en restauración, reservado). Para materiales digitales, gestiona el control de accesos concurrentes (número de licencias). Se integra con el Catalog Service para ofrecer una visión completa del fondo.
- **Microservicio de Circulación (Circulation Service):** El motor de préstamos y devoluciones. Gestiona todo el flujo de circulación de materiales: préstamos a domicilio, renovaciones, devoluciones, reservas de ejemplares prestados. Aplica las políticas de préstamo de la biblioteca (plazos según tipo de socio o material). Calcula automáticamente multas por devolución tardía y gestiona el bloqueo de socios morosos. Es un servicio crítico con alta concurrencia en horas punta.
- **Microservicio de Gestión de Adquisiciones (Acquisitions Service):** Apoya las tareas de desarrollo de la colección. Gestiona solicitudes de compra por parte de bibliotecarios o sugerencias de socios, pedidos a proveedores (distribuidoras, editoriales), recepción de materiales y su incorporación al catálogo. Mantiene un registro del presupuesto de adquisiciones y del gasto ejecutado.
- **Microservicio de Gestión de Salas y Espacios (Space Service):** Para bibliotecas que ofrecen salas de estudio, salas de reuniones o espacios de trabajo colaborativo. Permite a los socios **reservar espacios** en línea, consultar disponibilidad en tiempo real y gestionar su uso. Para el personal, ofrece herramientas para administrar la capacidad y las reglas de uso de cada espacio.
- **Microservicio de Portal del Socio (Patron Portal Service):** Un BFF (Backend for Frontend) específico para el portal web y la app móvil de los socios. Agrega datos de otros servicios (catálogo para búsquedas, circulación para ver préstamos activos, espacios para reservas) y los sirve en un formato optimizado. Gestiona la autenticación de los socios y su sesión.

- **Microservicio de Notificaciones (Notification Service):** Servicio transversal que envía comunicaciones a los socios: **recordatorios de devolución** (24h antes), **notificaciones de reserva disponible** (cuando un ejemplar reservado está listo para recoger), **avisos de multas**, y **comunicaciones generales** de la biblioteca. Se suscribe a eventos de otros servicios (préstamo creado, reserva activada) y envía emails, notificaciones push o SMS.
- **Microservicio de Informes y Estadísticas (Analytics Service):** Procesa los datos históricos de circulación, socios y adquisiciones para generar informes clave para la gestión bibliotecaria: **estadísticas de préstamos** (por materia, por tipo de socio), **rotación de la colección**, **socios más activos**, **uso de espacios**, y **cumplimiento del presupuesto**. Ayuda a la toma de decisiones sobre la colección y los servicios.

3. Arquitectura de Microservicios Detallada

1. **API Gateway:** Punto de entrada único para todas las clientes (portal web de socios, intranet de bibliotecarios, app móvil). Gestiona autenticación (JWT), rate limiting y enrutamiento.
2. **Patron Service:** Gestiona socios y personal. Es la fuente de verdad para identidad y roles.
3. **Catalog Service:** Gestiona el registro bibliográfico (la "ficha" del libro). Expone APIs de búsqueda avanzada (con filtros por materia, autor, etc.) y de gestión de autoridades.
4. **Holdings Service:** Gestiona los ejemplares físicos. Se relaciona con Catalog Service a través del bibliographic_record_id. Consulta frecuente para saber disponibilidad.
5. **Circulation Service:** El servicio de alta concurrencia. Gestiona préstamos, devoluciones, reservas y multas. Cada operación (préstamo) implica:
 - Verificar estado del socio (sin multas, activo) con Patron Service.
 - Verificar disponibilidad del ejemplar con Holdings Service (y actualizar su estado a "prestado").
 - Crear el registro de préstamo.
 - Publicar un evento loan.created. Toda esta lógica debe ser atómica o estar protegida por el patrón Saga.

6. **Acquisitions Service:** Gestiona pedidos a proveedores y sugerencias de compra. Menor concurrencia, pero con flujos de aprobación.
7. **Space Service:** Gestiona reservas de salas. Similar a un motor de reservas con disponibilidad en tiempo real.
8. **Patron Portal Service:** BFF para el frontend de socios. Simplifica las consultas agregando datos de Catalog, Holdings, Circulation y Space.
9. **Notification Service:** Escucha eventos (loan.created para recordatorio futuro, hold.available para aviso) y gestiona el envío.
10. **Analytics Service:** Ejecuta procesos ETL periódicos sobre réplicas de solo lectura de las bases de datos de otros servicios para generar informes sin afectar al rendimiento de los servicios transaccionales.

4. Estrategia de Base de Datos

Cada microservicio tendrá su propia base de datos **PostgreSQL** en Render.

- **Patron DB:** patrons (id, library_id, card_number, full_name, email, phone, address, registration_date, status, password_hash), staff (id, library_id, user_id, role, permissions), fines (id, patron_id, amount, reason, status, created_at).
- **Catalog DB:** bibliographic_records (id, title, uniform_title, edition, publication_year, publisher, isbn, issn, summary), authors (id, name, dates, authority_id), record_authors (record_id, author_id, role), subjects (id, term, authority_id), record_subjects (record_id, subject_id). Esta base de datos puede ser compleja debido a las relaciones MARC21.
- **Holdings DB:** items (id, bibliographic_record_id, barcode, call_number, collection, location (sala/estante), status (available, loaned, reserved, in_repair), acquisition_date, price). Para materiales digitales, digital_items (id, bibliographic_record_id, license_type, concurrent_users, file_url).
- **Circulation DB:** loans (id, item_id, patron_id, loan_date, due_date, return_date, status, renewed_count), reserves (id, item_id, patron_id, reserve_date, expiry_date, status), fines (id, loan_id, amount, status).
- **Acquisitions DB:** orders (id, supplier_id, order_date, status, total_amount), order_items (order_id, bibliographic_record_id, quantity, unit_price), suggestions (id, patron_id, title, author, reason, status, created_at).

- **Space DB:** spaces (id, name, type, capacity, location, amenities), space_reservations (id, space_id, patron_id, start_time, end_time, status, created_at).
- **Analytics DB:** Almacena datos pre-agregados para informes: monthly_loans_by_subject, patron_activity, collection_turnover.

5. Pila Tecnológica (Stack) Propuesta

Se propone **Node.js con NestJS** por su arquitectura modular y escalabilidad, ideal para un sistema con múltiples dominios de negocio bien definidos. Como alternativa, se valora **Python con FastAPI** por su claridad y rendimiento.

- **Backend (Microservicios - Opción Node.js):**
 - **Framework: NestJS** (sobre Node.js). Proporciona una arquitectura modular (cada microservicio como un módulo Nest), inyección de dependencias y soporte nativo para GraphQL, que podría ser útil para el portal del socio.
 - **ORM: Prisma** por su type-safety y facilidad para modelar relaciones complejas (como las del catálogo MARC21).
 - **Comunicación Asíncrona: Redis** con Pub/Sub para eventos.
 - **Búsqueda Avanzada:** Se podría integrar **Elasticsearch** junto al Catalog Service para ofrecer búsquedas rápidas y facetadas en el catálogo, sincronizando los datos desde PostgreSQL mediante eventos.
- **Backend (Microservicios - Opción Python):**
 - **Framework: FastAPI** para todos los servicios.
 - **ORM: SQLAlchemy** con **Alembic**.
 - **Comunicación Asíncrona: Redis** con Pub/Sub o **RabbitMQ**.
 - **Búsqueda Avanzada:** Integración con **Elasticsearch** mediante la librería elasticsearch-py.
- **Base de Datos y Caché:**
 - **Base de Datos Principal: PostgreSQL** (gestionado en Render).
 - **Caché/Broker de Eventos: Redis** (gestionado en Render).
 - **Motor de Búsqueda: Elasticsearch** (desplegado como servicio adicional en Render o usando un servicio gestionado externo).

- **Frontend:**

- **Portal del Socio (Web) e Intranet de Bibliotecarios: Next.js** (React) con **TypeScript** y **TailwindCSS**. El portal del socio necesita un buen SEO para ser encontrado por usuarios buscando libros. La intranet es una aplicación privada. Despliegue en **Vercel**.
- **App Móvil (Socios): React Native** para consultar el catálogo, renovar préstamos y reservar espacios desde el móvil. Despliegue en **App Store** y **Google Play Store**.

6. Comunicación entre Servicios

- **Síncrona (REST/GraphQL):** Para consultas inmediatas que requieren respuesta en el momento:
 - El portal del socio busca libros: la consulta va al Catalog Service (probablemente con GraphQL para flexibilidad) y puede enriquecerse con disponibilidad en tiempo real consultando al Holdings Service.
 - El bibliotecario registra un préstamo: la petición va al Circulation Service vía REST, que coordina la operación.
- **Asíncrona (Eventos con Redis):** Para desacoplar procesos secundarios y mantener la consistencia eventual:
 - Circulation Service publica loan.created tras un préstamo exitoso.
 - Notification Service consume loan.created para programar un recordatorio de devolución.
 - Analytics Service consume loan.created para actualizar sus métricas de uso.
 - Holdings Service podría actualizar algún contador de uso del ejemplar (aunque el estado ya lo cambió síncronamente).
- **Patrón Saga:** Para el proceso de "préstamo" que involucra múltiples servicios y debe ser consistente. Se podría implementar una saga coreografiada:
 1. Circulation Service inicia la saga y cambia el estado del préstamo a "pending".
 2. Llama (síncrono) a Patron Service para verificar que el socio no tiene multas. Si ok, continúa.

3. Llama a Holdings Service para reservar el ejemplar (cambiar estado a "prestado" temporalmente).
4. Si todo ok, confirma el préstamo. Si algo falla, ejecuta acciones de compensación (devolver ejemplar a disponible).

7. DevOps y Despliegue (CI/CD)

- **Control de Versiones:** Git con **GitHub Flow**.
- **Integración Continua (CI) con GitHub Actions:** Por cada Pull Request, pipeline por servicio con:
 - Linting y pruebas unitarias (Jest para NestJS, pytest para FastAPI).
 - **Pruebas de integración para Circulation Service**, simulando préstamos concurrentes.
 - Construcción de imagen Docker multi-stage.
- **Despliegue Continuo (CD):**
 - Al hacer merge a main, se construye la imagen final y se sube a **GHCR**.
 - Despliegue automático en **Render** (rolling updates).

8. Entorno de Despliegue y Seguridad

- **Frontend (Portal Socios / Intranet): Vercel.**
- **Backend, Bases de Datos y Elasticsearch:** Todos los microservicios, instancias de **PostgreSQL**, **Redis** y **Elasticsearch** en **Render**.
- **Seguridad:**
 - **Autenticación:** JWT para socios y personal.
 - **Autorización:** Middleware en cada servicio que verifica roles (ej. solo bibliotecarios pueden crear préstamos).
 - **Protección de Datos Personales:** Cumplimiento de normativas (GDPR) con cifrado de datos sensibles y gestión de consentimientos.
 - **Auditoría:** Registro de acciones críticas (préstamos, bajas de ejemplares) en tablas de log.
 - **Rate Limiting:** En el API Gateway para prevenir abusos en el catálogo público.

9. Observabilidad

- **Métricas (Prometheus):** Especial atención al Circulation Service: préstamos por minuto, tiempo de respuesta, tasa de éxito.
- **Dashboards (Grafana):** Visualización para bibliotecarios de métricas de uso (préstamos por hora, materiales más prestados) y para operaciones de la salud del sistema.
- **Logs:** Centralizados, con correlación de eventos de préstamo para trazabilidad.
- **Trazabilidad:** Implementación de tracing para seguir una solicitud de préstamo a través de Circulation, Patron y Holdings.